Columbia University

APMAE4302

Methods in computational science

Homework 3

Author:
Matías I. Inostroza
(mii2106)

March 9, 2026

1 Problem 1

1)

For the equation:

$$-\nabla^2 u + \gamma u^p = f \tag{1}$$

The exact solution is given by $u_{exact}(x, y) = A \exp\left(-\frac{(x-x_0)^2+(y-y_0)^2}{\sigma^2}\right)$. Accordingly, the source term is defined as $f = -\nabla^2 u_{exact} + \gamma u_{exact}^p$

The discretization is as follows:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u^{i-1,j} - 2u^{i,j} + u^{i+1,j}}{\Delta x^2} \tag{2}$$

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u^{i,j-1} - 2u^{i,j} + u^{i,j+1}}{\Delta y^2} \tag{3}$$

Thus, the equation becomes:

$$-\frac{u^{i-1,j} - 2u^{i,j} + u^{i+1,j}}{\Delta x^2} - \frac{u^{i,j-1} - 2u^{i,j} + u^{i,j+1}}{\Delta y^2} + \gamma(u^{i,j})^p = f \tag{4}$$

Assuming a uniform grid with spacing $\Delta x = \Delta y = h$, the discretized equation is given by

$$\frac{4u^{i,j} - u^{i-1,j} - u^{i+1,j} - u^{i,j-1} - u^{i,j+1}}{h^2} + \gamma(u^{i,j})^p = f \tag{5}$$

Accordingly, the residual at interior nodes is defined as:

$$F^{i,j}(u) = \frac{4u^{i,j} - u^{i-1,j} - u^{i+1,j} - u^{i,j-1} - u^{i,j+1}}{h^2} + \gamma(u^{i,j})^p - f = 0 \tag{6}$$

And for boundary nodes, the residual is given by:

$$F^{i,j}(u) = u^{i,j} - u_{exact}(x_i, y_i) = 0 \tag{7}$$

2) The Jacobian corresponds to the derivatives of the residual with respect to the nodal values of u. For interior nodes, the partial derivatives are given by:

$$\frac{\partial F^{i,j}}{\partial u^{i,j}} = \frac{4}{h^2} + \gamma p (u^{i,j})^{p-1} \tag{8}$$

$$\frac{\partial F^{i,j}}{\partial u^{i+1,j}} = \frac{\partial F^{i,j}}{\partial u^{i-1,j}} = \frac{\partial F^{i,j}}{\partial u^{i,j+1}} = \frac{\partial F^{i,j}}{\partial u^{i,j-1}} = -\frac{1}{h^2} \tag{9}$$

And for boundary nodes:

$$\frac{\partial F^{i,j}}{\partial u^{i,j}} = 1 \tag{10}$$

$$J(u) = 1 \tag{11}$$

If all unknowns are assembled into a global vector $\mathbf{u}$, the Jacobian associated with the interior equations can be written as:

$$\mathbf{J}(\mathbf{u}) = \mathbf{A} + \gamma p \, \mathrm{diag}\big(\mathbf{u}^{[p-1]}\big) \,, \tag{12}$$

where $\mathbf{A}$ is the five point finite difference matrix associated with the operator $-\nabla^2$ (each row as $\begin{bmatrix} ... & -1 & ... & -1 & 4 & -1 & ... & -1 & ... \end{bmatrix}$), and $\mathrm{diag}(\mathbf{u}^{[p-1]})$ is a diagonal matrix whose entries are u_k^{p-1}.

For boundary nodes, the corresponding rows of the Jacobian are replaced by rows of the identity matrix, since the boundary residual is simply $u_k - g_k = 0$. Thus, the global Jacobian is obtained by using the interior stencil rows for interior nodes and identity rows for boundary nodes.

3)

a) For this problem, the code `reaction2d.c` was developed and is attached in the repository.

b) The function f is defined as $f = -\nabla^2 u_{\text{exact}}$ when the option `-rct_linear_f` is set, and as $f = -\nabla^2 u_{\text{exact}} + \gamma u_{\text{exact}}^p$ otherwise, as implemented in the following code:

```
PetscReal fRHS(PetscReal x, PetscReal y, AppCtx *user) {
    PetscReal f = -d2ufunction(x,y);
    if (!user->linear_f) {
        PetscReal uex = ufunction(x, y);
        f += user->gamma * PetscPowReal(uex, (PetscReal)user
           ->p);
    }
    return f;
```

c) The residual is assembled using the function:

```
PetscErrorCode FormFunctionLocal(DMDALocalInfo *info,
                                 PetscReal **u,
                                 PetscReal **FF,
                                 AppCtx    *user) {
                                 .
                                 .
                                 .
                                 }
```

And the Jacobian is assembled using the function:

```
PetscErrorCode FormJacobianLocal(DMDALocalInfo *info,
                                 PetscReal    **u,
                                 Mat           J,
                                 Mat           P,
                                 AppCtx       *user) {
                                 .
                                 .
                                 .
                                 }
```

d) The Jacobian evaluation strategy can be selected through the following lines:

```
    PetscCall(DMDASNESSetJacobianLocal(da,
              (DMDASNESJacobianFn *)FormJacobianLocal, &user)
                 );
    PetscCall(SNESSetFromOptions(snes));
```

The call to `SNESSetFromOptions` allows PETSc to override the analytical Jacobian registered in `DMDASNESSetJacobianLocal` with finite-difference approximations using the options -snes_fd, -snes_fd_color, -snes_mf, and -snes_mf_operator at runtime, without modifying the code.

e) The ParaView output is generated using the following lines:

```
    PetscCall(PetscViewerVTKOpen(PETSC_COMM_WORLD, "
       reaction2d.vtr",
                                 FILE_MODE_WRITE, &viewer));
    PetscCall(PetscObjectSetName((PetscObject)u,      "u"));
    PetscCall(PetscObjectSetName((PetscObject)uexact, "uexact
       "));
    PetscCall(VecView(u,      viewer));
    PetscCall(VecView(uexact, viewer));
```

```
    PetscCall(DMView(da,      viewer));
    PetscCall(PetscViewerDestroy(&viewer));
```

The variable u corresponds to the numerical solution obtained using the finite difference method, while u_{exact} denotes the exact solution. No RHS is explicitly output, since the Newton Raphson method relies on residual evaluations rather than a RHS. This implementation is adapted from the `poisson2d.c` example.

4) For a 65×65 grid with $\gamma = 0$, the following result is obtained:

```
  0 SNES Function norm 7.337078385146e+02
    0 KSP Residual norm 7.337078385146e+02
    1 KSP Residual norm 6.204150044152e-11
  1 SNES Function norm 6.379557175902e-11
on 65 x 65 grid:  rel_error |u-uexact|_2/|uexact|_2 =
   0.000524515
```

The method converges in a single Newton iteration, reaching a residual norm of $\|F(u)\|_2 = 6.38 \times 10^{-11} < 10^{-10}$ and a relative error of 5.24×10^{-4}. The linear system at each Newton step is solved using direct LU factorization with `-ksp_type preonly -pc_type lu -pc_factor_mat_solver_type mumps`, requiring only one KSP iteration (see `options_file_gamma0` in the HW3 folder).

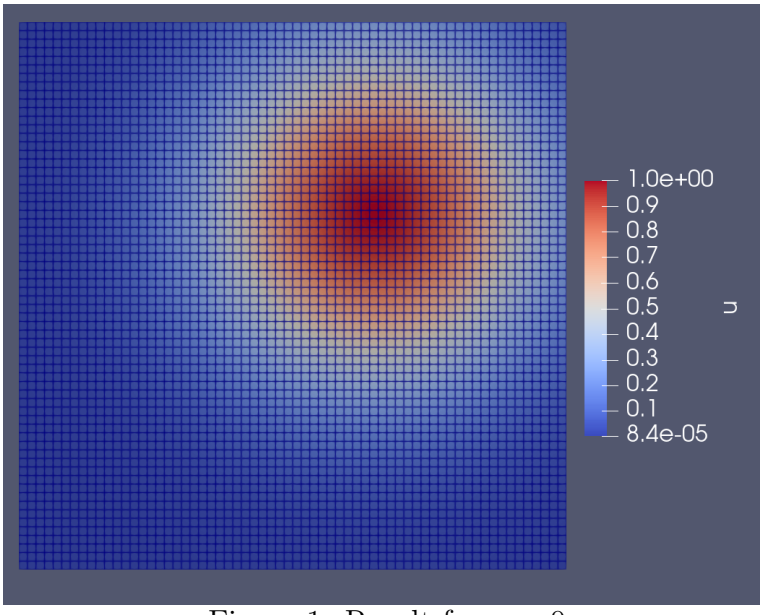

Figure 1: Result for $\gamma = 0$

Figure 1 shows the results from paraview.

5) After running the problem using finite difference Jacobians, the results listed in Table 1 are obtained:

Jacobian	Relative error	Function eval	Final Residual	Time (s)
Analytical	5.24×10^{-4}	2	6.38×10^{-11}	1.107×10^{-1}
FD (-snes_fd)	5.24×10^{-4}	4228	6.15×10^{-11}	1.430
FD Colored (-snes_fd_color)	5.24×10^{-4}	7		1.174×10^{-1}

Table 1: Performance comparison for different Jacobian strategies.

The analytical Jacobian significantly reduces the number of function evaluations compared to the standard finite difference approximation. The colored finite-difference approach achieves comparable performance to the analytical Jacobian, demonstrating efficiency.

6) For $\gamma = 100$ and $p = 3$, the nonlinear solver converges in 8 Newton iterations. The linear system at each Newton step is solved in a single KSP iteration. The nonlinear residual norms are shown in Table 2.

Newton Iteration k	$\|F(u^k)\|_2$	KSP Iterations
0	2.0978×10^3	-
1	1.8554×10^3	1
2	1.6274×10^3	1
3	1.2545×10^3	1
4	9.0808×10^2	1
5	9.4154×10^1	1
6	1.4429×10^0	1
7	3.5704×10^{-4}	1
8	3.9173×10^{-11}	1

Table 2: Nonlinear convergence history for $\gamma = 100$ and $p = 3$.

Figure 2 shows the behavior of the residual norm, exhibiting quadratic convergence.

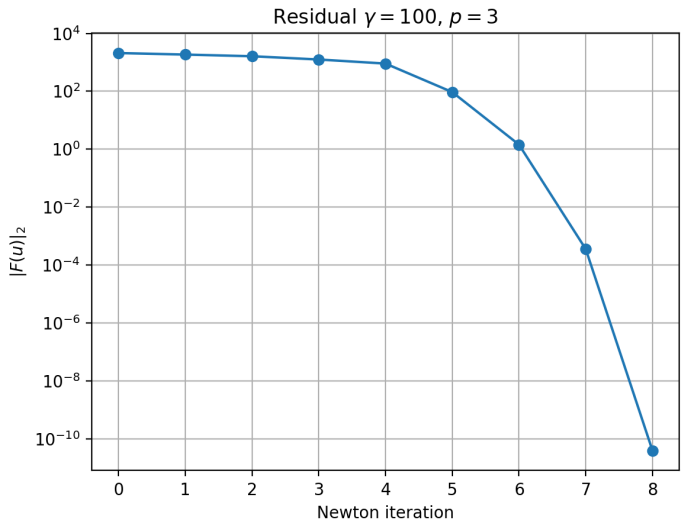

Figure 2: Convergence plot for $\gamma = 100$ and $p = 3$

7) Figure 3 shows the results for a linear case. The problem was run with the option `-rct_linear_f` for $\gamma = 100$ and $p = 3$ on a 65×65 grid. In this case, the right-hand side is defined as $f = -\nabla^2 u_{\text{exact}}$, omitting the reaction term $\gamma u_{\text{exact}}^p$.

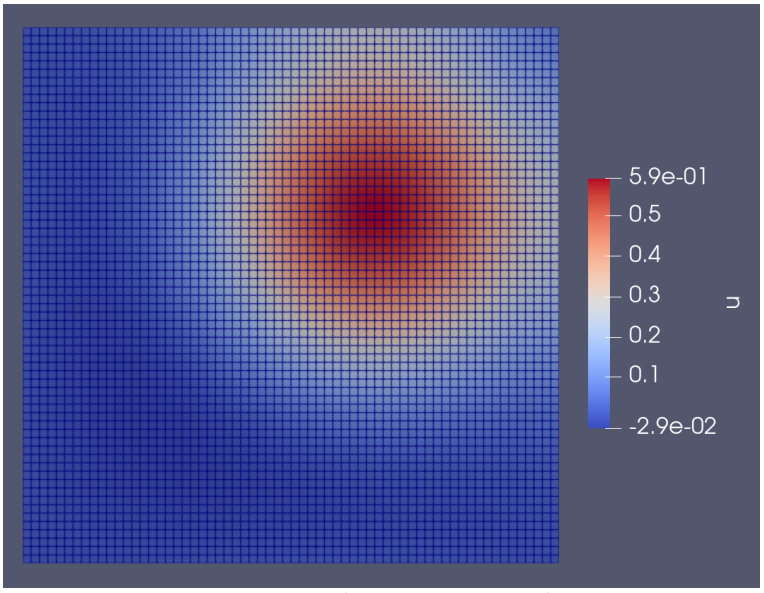

Figure 3: Results $\gamma = 100$ and $p = 3$

The obtained relative error was $\frac{\|u - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2} = 0.408$ which is significantly larger than in the full manufactured solution case (5.24×10^{-4}). The numerical solution reaches a maximum amplitude of approximately 0.59, compared to 1.0 for u_{exact}.

This occurs because using `-rct_linear_f` corresponds to solving the problem $-\nabla^2 u + \gamma u^p = -\nabla^2 u_{\text{exact}}$ which is not consistent with the manufactured solution, since the nonlinear term γu^p on the left-hand side has no corresponding term in the right-hand side. As a result, the solver converges to a solution different from u_{exact}.

Compared to the linear case $\gamma = 0$, the nonlinearity acts as an absorption term that reduces the amplitude of the solution. As γ increases, this effect becomes stronger, which explains the difference in amplitude between 0.59 and 1.0.

8) To run the code multiple times and collect the required data, a script `plot8.py` was developed with the assistance of Claude AI.

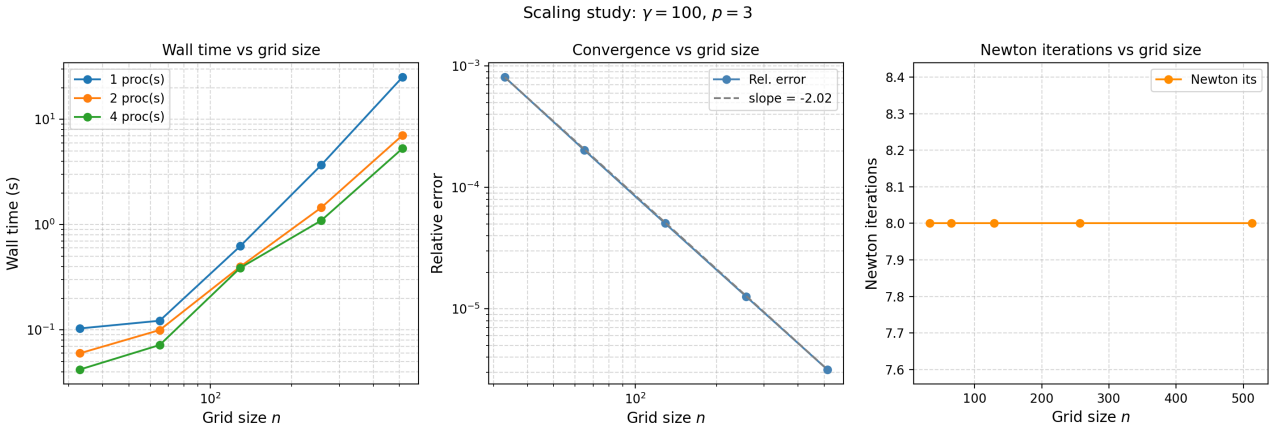

Figure 4: Plots for scaling and mesh refinement

Figure 4 and table 3 shows the performance and convergence behavior for $\gamma = 100$ and $p = 3$.

First, from the left panel (wall time vs grid size), the computational cost increases significantly with mesh refinement, as expected due to the quadratic growth in the number of unknowns. The runtime scales approximately as $\mathcal{O}(N)$ with respect to the number of degrees of freedom. Additionally, increasing the number of processors reduces the wall time, demonstrating parallel scalability. However, the speedup is sublinear, especially for finer grids, likely due to communication overhead and load imbalance.

From the middle panel (relative error vs grid size), the method exhibits a clear second-order convergence rate. The slope of approximately -2.02 confirms that the finite-difference discretization achieves the expected $O(h^2)$ accuracy. This validates the correctness of the implementation and the consistency of the numerical scheme.

Finally, the right panel (Newton iterations vs grid size) shows that the number of

n_{procs}	Refine	Grid	Newton Iter.	Time (s)	Rel. Error
1	2	33×33	8	1.032×10^{-1}	8.098×10^{-4}
1	3	65×65	8	1.221×10^{-1}	2.025×10^{-4}
1	4	129×129	8	6.240×10^{-1}	5.065×10^{-5}
1	5	257×257	8	3.679	1.267×10^{-5}
1	6	513×513	8	2.516×10^{1}	3.167×10^{-6}
2	2	33×33	8	5.996×10^{-2}	8.098×10^{-4}
2	3	65×65	8	9.959×10^{-2}	2.025×10^{-4}
2	4	129×129	8	3.981×10^{-1}	5.065×10^{-5}
2	5	257×257	8	1.450	1.267×10^{-5}
2	6	513×513	8	7.051	3.167×10^{-6}
4	2	33×33	8	4.204×10^{-2}	8.098×10^{-4}
4	3	65×65	8	7.178×10^{-2}	2.025×10^{-4}
4	4	129×129	8	3.882×10^{-1}	5.065×10^{-5}
4	5	257×257	8	1.093	1.267×10^{-5}
4	6	513×513	8	5.284	3.167×10^{-6}

Table 3: Strong scaling and mesh refinement results for the nonlinear solver.

Newton iterations remains constant (equal to 8) across all grid sizes. This could indicates that the nonlinear convergence is mesh independent and that Newton's method performs robustly for this problem. It also suggests that the initial guess and solver configuration are appropriate.

Overall, the results demonstrate that the solver achieves optimal convergence rates, good parallel scalability, and stable nonlinear performance across different grid resolutions.